Sahyog Care Companion: review guide

How to explain the architecture pack in plain words. Learn the first section by heart, and the rest follows from it.

The whole system in 30 seconds

> Sahyog is a hospital network with too many phone bookings and doctors losing time to typing. We built three things: patients find a doctor and book, patients ask questions about their own reports, and doctors get a draft note to review and sign. The main rule is that **the AI handles fuzzy steps, while code or a person makes anything that could harm a patient.**

The diagrams in plain words

Diagram	What it shows	Say this
Context	Who uses it and what it connects to	"Patients, caregivers, doctors and the front desk use it. It sends only scrubbed text to the AI model."
Container	The running pieces	"An API answers requests, and a separate worker does slow jobs like reading PDFs, so those can't slow down booking. Postgres stores the data, Redis is a short-term cache, and MinIO holds files."
Entity model	The database tables	"A patient books an appointment, which becomes a consultation, which produces a signed note. Reports are split into searchable pieces, each tagged with the patient's ID."
W1	Finding a doctor	"Check for an emergency first, then match a specialty, offer doctors, let the patient pick, and book."
W2 ingest	Uploading a report	"Read the PDF, check it belongs to this patient, scan for planted instructions, and store it as searchable pieces."
W2 ask	Asking about a report	"Check permission, check for an emergency, search only this patient's pieces, answer with page numbers, then have code check the answer."
W3	The doctor's note	"Turn speech into a draft, flag stock, then stop and wait. Nothing is saved until the doctor signs."
Sequences	The same flows, message by message	"The graph saves its progress and stops while waiting for a person, then resumes from there."

Five words you'll need

- **Graph:** a flowchart the code follows, with steps and branches.
- **Interrupt:** the graph pauses and waits for a person.
- **Checkpoint:** the saved progress that lets it resume later.

- **Scrubber:** removes names and phone numbers before text goes to the AI.
- **Red flag:** a sign of an emergency, like chest pain.

The reviewer's eight questions (pack page 50)

1. **Why a graph, not a chain of calls?** "We need to pause for a doctor's signature and resume later. A plain chain can't wait hours and pick up where it stopped."
2. **How can a patient never see another's data?** "The patient filter is in the database query, and the ID comes from their login, not from the AI. A poisoned PDF can't change that query. Our red-team test confirms it."
3. **What if the model API is down?** "Each call has a timeout and a retry, then a fallback. After 5 failures in a row we stop calling that provider for 30 seconds. If it still fails, the patient goes to the front desk."
4. **How do you know a cited answer is right?** "Code checks that each citation belongs to this patient and that every number appears on the cited page. Our AI judge isn't calibrated yet, and that's a known gap."
5. **On-premise or cloud?** "Hybrid. Patient data stays inside, and only scrubbed text goes out. ADR 0005 only proposes this, and nothing is deployed."
6. **What did the change request cost?** *No record of this in the repo. Ask your pod or check the client thread. Don't guess in the review.*
7. **What does a consultation cost?** "About $0.017 in model cost for one triage, one answer and one note. Infrastructure isn't priced yet."
8. **Where did your coding agent mislead you?** "The scrubber it built also hid lab test names from the model, which could cause a wrong 'not found'. We caught it and added test F22."

Weak spots to admit first

Reviewers respect this more than finding it themselves.

- The independent emergency test has never run, and it's the biggest risk.
- There's no rate limiting.
- It has only run on one Mac, and triage narrowly missed its speed target (p95 4.28 s against 4 s).
- Everything is synthetic data, and no clinician has reviewed it.

Know these three especially well

The reviewer picks one node, one decision and one failure from the repo.

- **Node: `safety_check`.** Before any answer reaches the patient, code checks that every citation belongs to them, every number appears in the cited page, and there's no diagnosis or dose wording. It also adds the scanned-page warning itself.
- **Decision: the SQL patient filter.** The patient ID comes from the login, and the filter sits in the database query, so no prompt can widen it.

- **Failure: "Known diabetic" heard as "NON diabetic".** Speech-to-text got it wrong, and our grounding check trusts the transcript, so it couldn't catch it. The doctor's review is the safeguard.

> Practise by running the demo script at `http://localhost:8000/` and saying each step aloud as you click.

Source: docs/architecture-pack.md, docs/FINAL-REPORT.md and the stage records in the sahyog-care-companion repository. Synthetic data only.